UNIVERSITÀ DEGLI STUDI DI TRIESTE

Birch clustering

Implementation by Poornima Devi Krishnasamy Karthikeyan

https://dl.acm.org/doi/pdf/10.1145/235968.233324

Unsupervised Machine Learning

Contents:

- Introduction
- Motivation
- How it works?
- Implementation
  - Comparison of results
- Conclusion

Introduction :

Idea :

- Clustering - To group data points that are similar and try to keep them as far away from things that are dissimilar.

  Problem:
- Most clustering algorithms has problems with processing large datasets with limited amount of resources (i.e. memory and CPU).

Motivation- Birch Clustering:

- Minimize running time and data scans, thus formulating the problem for large databases.
- Creating a summary of data while retaining much of the distribution related information and then clustering the summary.
- Exploit the non uniformity of data – treat dense areas as one, and remove outliers (noise)
- Balanced iterative reducing and clustering using hierarchies

How it works?

- Phase 1: Scan all data and build an initial in-memory CF tree.
- Phase 2: condense into desirable length by building a smaller CF tree.
- Phase 3: Global clustering
- Phase 4: Cluster refining – this is optional, and requires more passes over the data to refine the results

Clustering feature (CF) :

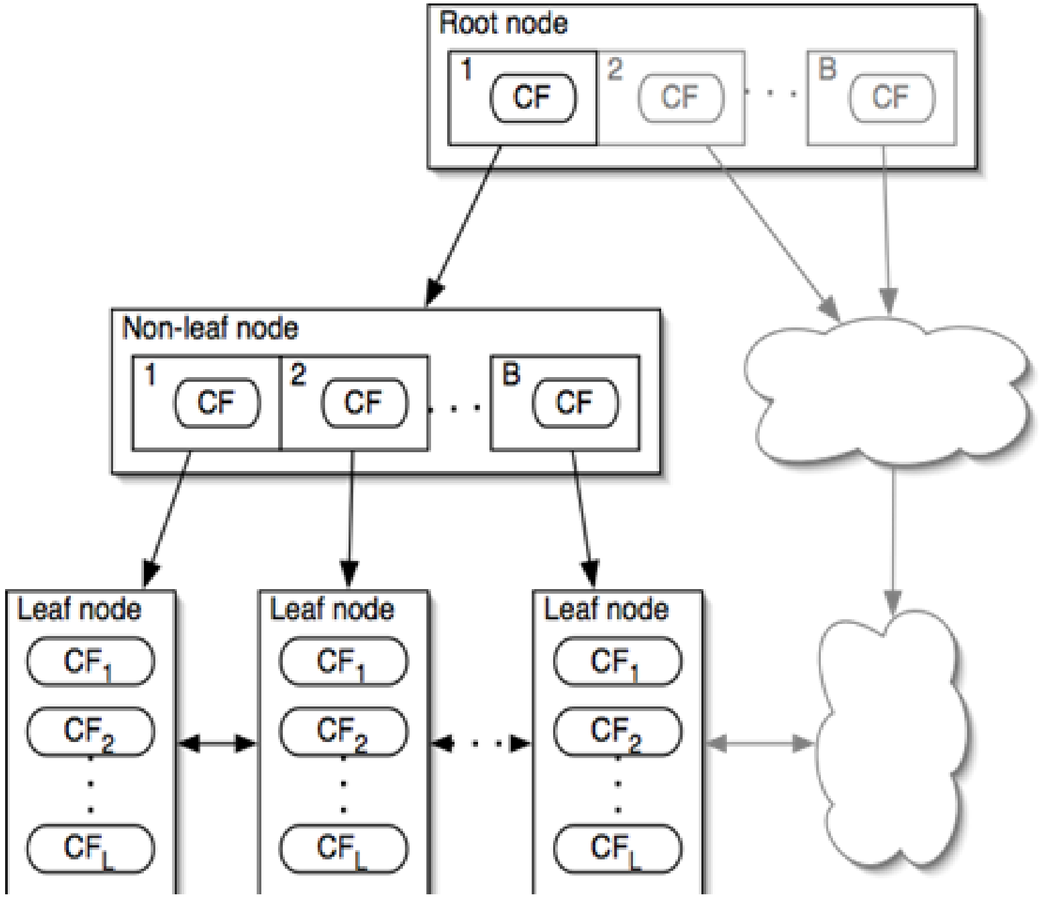
- CF = (N, LS, SS)
  - N - number of data points in the cluster.
  - LS - linear sum of the N data points $\sum_{i=1}^{N} X_i$
  - SS - squared sum of the N data points $\sum_{i=1}^{N} X_i^2$

- They can be used to compute other essential values for clustering, centroid, radius, diameter, average link distance between clusters.
- CFmerge = CF1 + CF2 = (N1 + N2 , LS1 + LS2, SS1 + SS2)

Clustering Feature Tree :

Hyperparameters:

- T: Cluster diameter threshold for leaf entries
- B : Branching factor, length of an internal node
- L: length of a leaf node
- B,L depends on the memory page size.

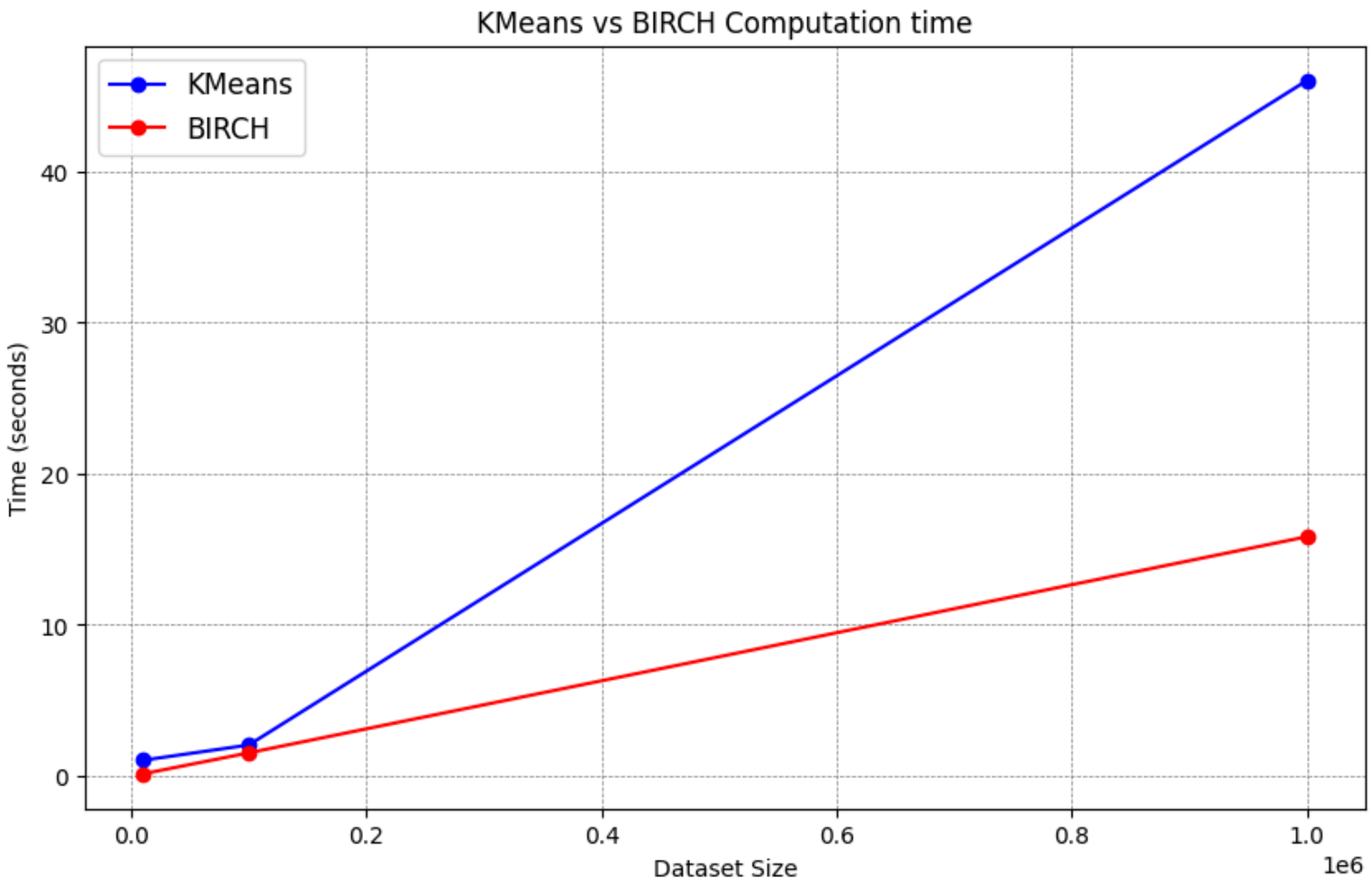


CF tree holds CF vectors of clusters but not raw data

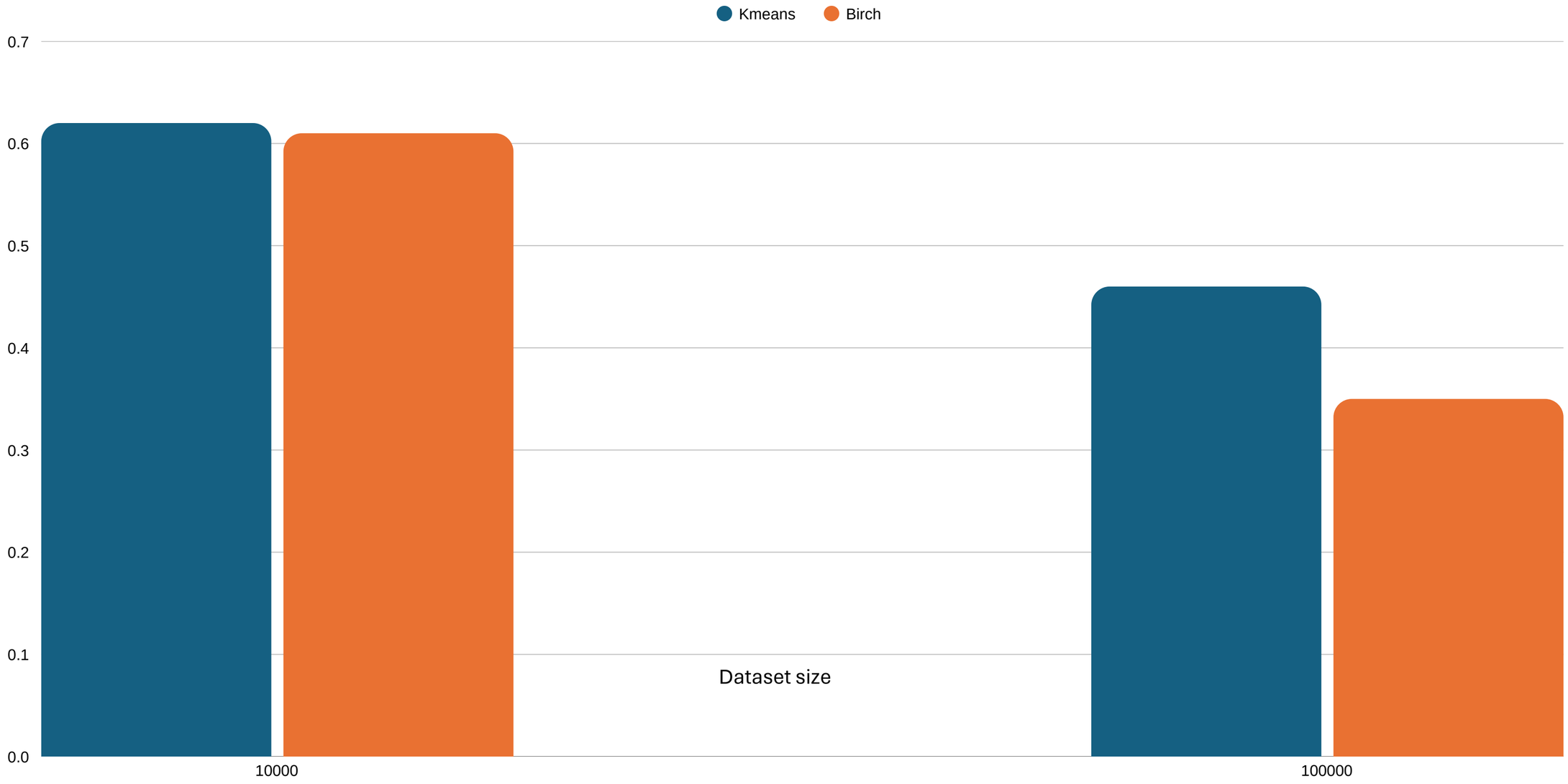
Implementation:

- Created 3 synthetic data sets for testing.
  - of sizes 10000,100000,1000000.
- KMEANS require entire data set to be in memory
  - *Initial scan is from disk, subsequent scans are in memory*

Implementation – Results:



Implementation – Results:



Conclusion

- Birch performs faster than existing algorithms (like KMEANS) on large datasets in *speed, stability and scalability*
- Scans whole data only once
- Handles outliers better
- Cons : only takes numerical data

Pros and Cons of Birch algorithm:

Pros:

- Efficient for clustering tasks that can’t fit into memory.
- Different clustering algorithms can be used on the ‘tight’ clusters in the leaf nodes
- Removes outliers

Cons:

- Only work with numerical data